

Báo cáo cs112 - Geometric Algorithms

Ngày 19 tháng 11 năm 2025

1 Lời giải Subtask 1

1.1 Phân tích và lựa chọn phương pháp thiết kế thuật toán

Trong Subtask 1, ta có các ràng buộc:

$$n, m \leq 1000.$$

Mỗi vùng theo dõi Aura được biểu diễn bằng một hình chữ nhật không giao nhau. Mỗi Hunter có một vị trí (C_j, R_j) và lượng Aura t_j . Nhiệm vụ là xác định mỗi Hunter thuộc vùng theo dõi nào (nếu có) và cộng tổng Aura vào vùng đó.

Với giới hạn nhỏ của Subtask 1, phương pháp phù hợp nhất là:

Duyệt tuần tự toàn bộ các vùng cho mỗi Hunter (Brute-force).

1.2 Vì sao phương pháp này phù hợp?

- Trong Subtask 1, số lượng vùng và số lượng Hunter đều không vượt quá 1000, vì vậy số phép kiểm tra tối đa là:

$$n \times m \leq 10^6,$$

hoàn toàn có thể xử lý trong thời gian rất ngắn.

- Các vùng không giao nhau và không chạm nhau, nên mỗi Hunter chỉ có thể nằm trong *nhiều nhất một* vùng. Điều này cho phép ta dừng kiểm tra ngay khi tìm thấy vùng phù hợp.
- Không cần dùng đến cấu trúc dữ liệu phức tạp như sweep line hay segment tree, giúp lời giải đơn giản, rõ ràng và dễ kiểm chứng.

Vì các lý do trên, **thuật toán duyệt tuyến tính toàn bộ vùng cho mỗi Hunter** là phương pháp tối ưu cho Subtask 1.

1.3 Mô tả thuật toán

Với mỗi Hunter có tọa độ (C_j, R_j) , ta duyệt qua tất cả các vùng hình chữ nhật:

$$(X_i, Y_i) \text{ đến } (A_i, B_i),$$

và kiểm tra điều kiện bao chứa:

$$X_i \leq C_j < A_i \quad \text{và} \quad Y_i \leq R_j < B_i.$$

- Nếu Hunter thuộc vùng i , cộng giá trị t_j vào vùng i và dừng duyệt.
- Nếu không vùng nào chứa Hunter, ta cộng vào phần tử thứ $(n+1)$, tức vùng “không được theo dõi”.

1.4 Độ phức tạp thời gian

Do duyệt toàn bộ m Hunter và kiểm tra qua toàn bộ n vùng:

$$T(n, m) = O(nm).$$

Với $n, m \leq 1000$:

$$T = 10^6 \text{ phép kiểm tra,}$$

một con số rất nhỏ, chạy dưới vài phần nghìn giây.

1.5 Độ phức tạp không gian

Ta chỉ lưu:

- danh sách n vùng hình chữ nhật,
- mảng kết quả kích thước $n+1$.

Do đó:

$$\text{Không gian} = O(n),$$

không đáng kể.

1.6 Kết luận cho Subtask 1

Phương pháp **Brute-force Checking** toàn bộ vùng cho mỗi Hunter là hoàn toàn phù hợp và tối ưu cho Subtask 1 vì:

- đơn giản,
- đúng bản chất bài toán,
- đủ nhanh cho giới hạn nhỏ,
- không yêu cầu cấu trúc dữ liệu nâng cao.

Dưới đây là mã minh họa cho Subtask 1.

```
def solve():
    import sys
    data = list(map(int, sys.stdin.read().split()))
    it = iter(data)

    W = next(it)
    H = next(it)
    n = next(it)
    m = next(it)

    rects = []
    for _ in range(n):
        x1 = next(it)
```

```

        y1 = next(it)
        x2 = next(it)
        y2 = next(it)
        if x1 > x2: x1, x2 = x2, x1
        if y1 > y2: y1, y2 = y2, y1
        rects.append((x1, y1, x2, y2))

ans = [0] * (n + 1)

for _ in range(m):
    cx = next(it)
    cy = next(it)
    t = next(it)

    inside = False
    for i in range(n):
        x1, y1, x2, y2 = rects[i]
        if x1 <= cx < x2 and y1 <= cy < y2:
            ans[i] += t
            inside = True
            break

    if not inside:
        ans[n] += t

print(" ".join(map(str, ans)))

```

2 Lời giải Subtask 2

2.1 Phân tích đặc trưng của Subtask 2

Điểm then chốt:

- Kích thước mặt phẳng nhỏ ($W \cdot H \leq 10^6$),
- Nhưng số vùng và số Hunter lớn (10^5),
- Các vùng Aura không chồng nhau và không chạm nhau.

Điều này cho phép xây dựng biểu diễn rời rạc của toàn bộ mặt phẳng (grid) với chi phí chấp nhận được.

2.2 Phương pháp thiết kế thuật toán được lựa chọn: Transform and Conquer

Nhóm **Transform and Conquer** bao gồm các kỹ thuật như:

- thay đổi biểu diễn dữ liệu (Representation Change),
- biến đổi bài toán sang dạng dễ xử lý hơn,
- rời rạc hóa (Discretization),

- rasterization.

Thuật toán Subtask 2 áp dụng chính xác dạng **Representation Change**:

1. Ban đầu bài toán là kiểm tra điểm nằm trong hình chữ nhật bất kỳ.
2. Ta biến đổi toàn bộ mặt phẳng thành một lưới rời rạc $W \times H$.
3. Mỗi ô (x, y) của lưới được gán một giá trị id biểu thị vùng Aura mà ô thuộc vào (hoặc -1 nếu không thuộc vùng nào).
4. Nhờ đó, việc truy vấn Hunter chuyển từ kiểm tra hình học sang đọc một ô của mảng 2D: $id[C_j][R_j]$.

Sự biến đổi này giảm mạnh độ phức tạp truy vấn và là lý do thuật toán được xếp vào nhóm **Transform and Conquer**.

2.3 Mô tả thuật toán chi tiết

1. Tạo ma trận id kích thước $W \times H$, gán toàn bộ bằng -1 .
2. Với mỗi vùng Aura i có hai góc (X_i, Y_i) và (A_i, B_i) , tô:

$$\forall x \in [X_i, A_i - 1], \forall y \in [Y_i, B_i - 1], \quad id[x][y] = i.$$

Vì các vùng không giao nhau nên không cần xử lý ghi đè.

3. Với mỗi Hunter (C_j, R_j, t_j) :
 - Nếu $id[C_j][R_j] = -1$: cộng t_j vào vùng ngoài cùng (vị trí $n + 1$),
 - Nếu không: cộng t_j vào vùng được chỉ định bởi id .
4. Xuất kết quả.

2.4 Phân tích độ phức tạp thời gian

- Tô lưới: tối đa $W \cdot H$ ô, suy ra:

$$O(W \cdot H) \leq O(10^6)$$

- Xử lý Hunter: mỗi Hunter chỉ truy cập $O(1)$:

$$O(m) \leq 10^5$$

- Tổng độ phức tạp:

$$T = O(W \cdot H + m)$$

2.5 Phân tích độ phức tạp không gian

- Lưới id: $W \cdot H \leq 10^6$ ô:

$$O(W \cdot H)$$

- Mảng kết quả: $O(n)$

- Tổng không gian:

$$S = O(W \cdot H + n)$$

Dưới đây là mã minh họa cho Subtask .

```
#!/usr/bin/env python3
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    it = iter(map(int, data))

    try:
        W = next(it)
        H = next(it)
        n = next(it)
        m = next(it)
    except StopIteration:
        return

    # Quick sanity: if W*H too large, stop (this solution assumes
    # W*H <= ~1e6)
    if W <= 0 or H <= 0:
        print(" ".join(["0"] * (n + 1)))
        return

    if W * H > 5_000_000:
        # safety guard: still attempt but warn (can be removed)
        pass

    # grid[x][y] = region id (0..n-1), -1 means outside all
    # regions
    # We'll store as list of lists for simplicity: grid[x][y]
    # If memory/caching prefer row-major grid[y][x], but both
    # work for limits here.
    grid = [[-1] * H for _ in range(W)]

    rects = []
    for idx in range(n):
        try:
            x1 = next(it); y1 = next(it); x2 = next(it); y2 =
                next(it)
        except StopIteration:
            x1 = y1 = x2 = y2 = 0
```

```

# normalize
if x1 > x2:
    x1, x2 = x2, x1
if y1 > y2:
    y1, y2 = y2, y1
# clamp to grid bounds (optional, but safe)
if x1 < 0: x1 = 0
if y1 < 0: y1 = 0
if x2 > W: x2 = W
if y2 > H: y2 = H
rects.append((x1, y1, x2, y2))
# fill grid: [x1, x2) x [y1, y2)
for x in range(x1, x2):
    row = grid[x]
    for y in range(y1, y2):
        row[y] = idx

ans = [0] * (n + 1)

for _ in range(m):
    try:
        cx = next(it); cy = next(it); t = next(it)
    except StopIteration:
        break
    # if hunter outside map, count as outside
    if not (0 <= cx < W and 0 <= cy < H):
        ans[n] += t
    else:
        rid = grid[cx][cy]
        if rid == -1:
            ans[n] += t
        else:
            ans[rid] += t

# print n+1 numbers
sys.stdout.write(" ".join(map(str, ans)))

if __name__ == "__main__":
    solve()

```

3 Lời giải Subtask 3

3.1 Đặc trưng của Subtask 3

Ở Subtask 3, ràng buộc lớn hơn nhiều:

$$W, H \leq 10^9, \quad n, m \leq 10^5.$$

Kích thước không gian rất lớn nên không thể rời rạc hoá toàn bộ mặt phẳng thành lưới như Subtask 2. Cần một phương pháp có khả năng xử lý 100k vùng và 100k Hunter mà không phụ thuộc vào kích thước bản đồ.

3.2 Phương pháp thiết kế thuật toán được sử dụng: Transform and Conquer

Subtask 3 sử dụng thuật toán **Sweep line**, phương pháp thiết kế thuật toán **Transform and Conquer**

Cụ thể thuộc dạng:

- **Representation Change**: biến đổi biểu diễn hình chữ nhật sang khoảng (interval) trên trục y sau khi nén tọa độ.
- **Input Transformation**: chuyển toàn bộ bài toán 2D sang chuỗi sự kiện 1D theo trục x (event-based formulation).

3.3 Ý tưởng thuật toán

1. Thu thập tất cả y từ các vùng Aura.
2. Thực hiện **Coordinate Compression** tạo ra trục y rời rạc có kích thước $O(n)$.
3. Mỗi vùng Aura trở thành một đoạn $[L_i, R_i)$ trên trục y sau nén.
4. Chuyển toàn bộ bài toán thành danh sách sự kiện theo x :
 - (x_1, enter, i) : thêm vùng i khi quét đến x_1 ,
 - (x_2, exit, i) : xoá vùng i khi đi qua x_2 ,
 - (C_j, query, j) : xử lý Hunter thứ j tại tọa độ C_j .
5. Sắp xếp các sự kiện theo trục x .
6. Dùng cấu trúc dữ liệu Segment Tree dạng “stack per node” để lưu vùng Aura đang hoạt động.
7. Khi gặp sự kiện query:

$\text{region} = \text{seg.query}(R_j)$

rồi cộng lượng Aura tương ứng.

3.4 Độ phức tạp thời gian

$$O((n + m) \log n)$$

Giải thích:

- Nén tọa độ: $O(n \log n)$
- Sắp xếp sự kiện: $O((n + m) \log(n + m))$
- Mỗi update/query segment tree: $O(\log n)$

3.5 Độ phức tạp không gian

$O(n)$

bao gồm:

- mảng tọa độ nén,
- Segment Tree rời rạc,
- danh sách sự kiện.

Dưới đây là mã minh họa cho Subtask 3.

```
import sys
from bisect import bisect_left, bisect_right

class SparseSegStacks:
    __slots__ = ("n", "size", "stacks")
    def __init__(self, n_segments: int):
        self.n = n_segments
        size = 1
        while size < n_segments:
            size <<= 1
        self.size = size
        self.stacks = {}

    def push_range(self, l: int, r: int, rid: int):
        if l >= r or self.n == 0:
            return
        size = self.size
        stacks = self.stacks
        l += size
        r += size
        while l < r:
            if l & 1:
                if l not in stacks:
                    stacks[l] = [rid]
                else:
                    stacks[l].append(rid)
                l += 1
            if r & 1:
                r -= 1
                if r not in stacks:
                    stacks[r] = [rid]
                else:
                    stacks[r].append(rid)
            l >>= 1
            r >>= 1

    def pop_range(self, l: int, r: int):
        if l >= r or self.n == 0:
            return
        size = self.size
```

```

        stacks = self.stacks
        l += size
        r += size
        while l < r:
            if l & 1:
                st = stacks.get(l)
                st.pop()
                if not st:
                    del stacks[l]
                l += 1
            if r & 1:
                r -= 1
                st = stacks.get(r)
                st.pop()
                if not st:
                    del stacks[r]
            l >>= 1
            r >>= 1

    def query_index(self, idx: int):
        if idx < 0 or idx >= self.n:
            return None
        node = idx + self.size
        stacks = self.stacks
        while node:
            st = stacks.get(node)
            if st:
                return st[-1]
            node >>= 1
        return None

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    it = iter(data)
    try:
        W = next(it)
        H = next(it)
        n = next(it)
        m = next(it)
    except StopIteration:
        return

    x1 = [0]*n
    y1 = [0]*n
    x2 = [0]*n
    y2 = [0]*n
    Ys = []

    for i in range(n):
        a = next(it)
        b = next(it)

```

```

        c = next(it)
        d = next(it)
        if a > c:
            a, c = c, a
        if b > d:
            b, d = d, b
        x1[i], y1[i], x2[i], y2[i] = a, b, c, d
        Ys.append(b)
        Ys.append(d)

px = [0]*m
py = [0]*m
pt = [0]*m

for j in range(m):
    px[j] = next(it)
    py[j] = next(it)
    pt[j] = next(it)

Ys.sort()
k = 1
for i in range(1, len(Ys)):
    if Ys[i] != Ys[k-1]:
        Ys[k] = Ys[i]
        k += 1
Ys = Ys[:k]

Nseg = max(0, len(Ys) - 1)
seg = SparseSegStacks(Nseg)

L = [0]*n
R = [0]*n
for i in range(n):
    L[i] = bisect_left(Ys, y1[i])
    R[i] = bisect_left(Ys, y2[i])

leaf_idx = [0]*m
for j in range(m):
    leaf_idx[j] = bisect_right(Ys, py[j]) - 1

events = []
add = events.append

for i in range(n):
    add((x1[i], 1, i))
    add((x2[i], 0, i))
for j in range(m):
    add((px[j], 2, j))

events.sort()

```

```

ans = [0]*(n+1)

for x, typ, idx in events:
    if typ == 1:
        seg.push_range(L[idx], R[idx], idx)
    elif typ == 0:
        seg.pop_range(L[idx], R[idx])
    else:
        j = idx
        kj = leaf_idx[j]
        t = pt[j]
        if kj < 0 or kj >= Nseg:
            ans[n] += t
        else:
            rid = seg.query_index(kj)
            if rid is None:
                ans[n] += t
            else:
                ans[rid] += t

sys.stdout.write(" ".join(map(str, ans)))

if __name__ == "__main__":
    main()

```